

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

Common Errors

Anti-patterns and mistakes to watch for when creating or reviewing tasks.

Instruction Problems

Feedback categories: `instruction_styling`, `expose_answers`

Ambiguous Language

AVOID	USE INSTEAD
"Make it better"	"Reduce runtime by 50%"
"Fix the issues"	"Fix the 3 failing tests"
"Handle errors properly"	"Return HTTP 400 for invalid input"
"Optimize the code"	"Achieve $O(n \log n)$ complexity"

Relative Paths

Bad - `instruction.md`:

MARKDOWN

 Copy

Edit `config/settings.json`

Good - `instruction.md`:

MARKDOWN

 Copy

```
Edit `/app/config/settings.json`
```

Missing Output Specs

Bad - `instruction.md`:

MARKDOWN

 Copy

```
Process the data and save results.
```

Good - `instruction.md`:

MARKDOWN

 Copy

```
Process `/data/input.csv` and save results to `/output/results.json`
```

Unverifiable Tool Requirements

Bad - `instruction.md` (can't verify vim was used):

MARKDOWN

 Copy

```
Use vim to edit the file.
```

Good - `instruction.md` (verifies the result):

MARKDOWN

 Copy

```
Change the port from 8080 to 3000 in `/app/config.txt`
```

Test Problems

Feedback categories: [test_alignment](#), [test_build](#)

Brittle String Matching

PYTHON

 Copy

```
# Bad - Breaks with formatting changes
def test_output():
    output = open("/output/log.txt").read()
    assert output == "Processing complete.\n"

# Good - Checks for key content
def test_output():
    output = open("/output/log.txt").read()
    assert "complete" in output.lower()
```

Implementation Testing

PYTHON

 Copy

```
# Bad - Tests implementation, not behavior
def test_uses_sorted():
    code = open("/app/main.py").read()
    assert "sorted(" in code

# Good - Tests actual behavior
def test_output_is_sorted():
    result = process_data()
    assert result == sorted(result)
```

Missing Docstrings

PYTHON

 Copy

```
# Bad
def test_1():
```

```
    assert process("") == []

# Good
def test_empty_input_returns_empty_list():
    """Verify that empty string input returns an empty list."""
    assert process("") == []
```

Order-Dependent Tests

PYTHON

 Copy

```
# Bad - test_2 depends on test_1
def test_1_setup():
    global data
    data = load_data()

def test_2_process():
    result = process(data)
    assert result

# Good - Each test is independent
def test_process():
    data = load_data()
    result = process(data)
    assert result
```

Hardcoded Random Values

PYTHON

 Copy

```
# Bad - Assumes specific random output
def test_random_sample():
    result = random_sample(data)
    assert result == [3, 7, 2]

# Good - Tests properties
def test_random_sample_size():
    result = random_sample(data, n=3)
```

```
assert len(result) == 3
assert all(item in data for item in result)
```

Solution Problems

Feedback category: [oracle](#)

Hardcoded Answers

BASH Copy

```
# Bad - Just outputs the answer
echo "The answer is 42" > /output/result.txt

# Good - Derives the answer
python /app/calculate.py > /output/result.txt
```

Non-Deterministic Commands

BASH Copy

```
# Bad - Output order varies
ls /data/ > /output/files.txt

# Good - Consistent ordering
ls /data/ | sort > /output/files.txt
```

Missing Error Handling

BASH Copy

```
# Bad - Continues after errors
cd /nonexistent
do_something

# Good - Fails fast
```

```
set -e
cd /app
do_something
```

Environment Problems

Feedback category: `environment` (distinct from `pinning`, which covers version pins specifically)

Missing Agent Runtime Dependencies

The agent runtime requires `tmux` and `asciinema` in every task image. Missing these causes all agent runs to fail silently before any verifier output is produced — the symptom is `RuntimeError: Failed to start tmux session` with `verifier_did_not_run` across every trial.

DOCKERFILE

[Copy](#)

```
# Bad - Missing tmux and asciinema
FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm
WORKDIR /app
RUN pip install pandas==2.0.0

# Good - Pre-installs the required agent runtime deps
FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm
WORKDIR /app
RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        tmux \
        asciinema \
    && rm -rf /var/lib/apt/lists/*
RUN pip install pandas==2.0.0
```

Runtime Network Installs in test.sh

With `allow_internet = false`, `tests/test.sh` cannot fetch packages from the network at runtime. All verifier dependencies must be baked into the Docker image. A `test.sh` that runs `apt-get install`, `curl ...`

`install.sh`, `uvx`, `pip install`, `npm install`, `git clone`, or `wget` will succeed locally during development (where the network is available) and fail in production with `RewardNotFoundError`.

BASH Copy

```
# Bad - test.sh installs deps at runtime, fails with allow_internet
#!/bin/bash
apt-get update && apt-get install -y curl
curl -LsSf https://astral.sh/uv/0.9.5/install.sh | sh
uvx -w pytest==8.4.1 pytest /tests/test_outputs.py -rA

# Good - test.sh assumes deps already in image
#!/bin/bash
python -m pytest --ctrf /logs/verifier/ctrf.json /tests/test_output
rc=$?
if [ "$rc" -eq 0 ]; then
    echo 1 > /logs/verifier/reward.txt
else
    echo 0 > /logs/verifier/reward.txt
fi
```

And the corresponding `Dockerfile` line that makes this work:

DOCKERFILE Copy

```
# In environment/Dockerfile
RUN pip install --no-cache-dir pytest==8.4.1 pytest-json-ctrf==0.3.
```

AI-Framework Scaffolding Filenames

Filenames like `CLAUDE.md`, `skills.md`, `AGENTS.md`, or `.cursor/` directories indicate the task was generated or assisted by an AI framework and not cleaned up before submission. They raise authenticity concerns and must be removed from the task environment.

TEXT



```
# Bad - Scaffolding files left in the environment
my-task/
├─ environment/
│   ├── Dockerfile
│   ├── CLAUDE.md          ← Remove
│   ├── skills.md          ← Remove
│   ├── .cursor/           ← Remove
│   └─ app/
│       └─ main.py

# Good - Only task-specific, professionally named files
my-task/
├─ environment/
│   ├── Dockerfile
│   ├── README.md          ← Real project documentation is fine
│   └─ app/
│       └─ main.py
```

Solution or Tests Copied Into the Image

The `solution/` and `tests/` directories are copied into the container by Harbor at runtime (`/oracle/` and `/tests/` respectively). Copying them in the `Dockerfile` either bakes the answers into the image (where agents could read them) or shadows the runtime mount and breaks verification entirely.

DOCKERFILE



```
# Bad - Bakes the solution and tests into the image
FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm
WORKDIR /app
COPY . /app/                # Copies solution/ and tests/
RUN pip install pandas==2.0.0

# Good - Only copy the agent-facing files
FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm
WORKDIR /app
```



```
COPY app/ /app/           # Only the project under test
RUN pip install pandas==2.0.0
```

Privileged Mode and Dangerous Capabilities

Tasks must not require `--privileged`, capabilities like `SYS_ADMIN` / `NET_ADMIN` / `SYS_MODULE`, or mounts of `/var/run/docker.sock`. These create security exposure and don't reproduce in the production verifier sandbox.

YAML

[Copy](#)

```
# Bad - docker-compose.yaml grants dangerous capabilities
services:
  app:
    image: my-task-base:1.0
    privileged: true
    cap_add:
      - SYS_ADMIN
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock

# Good - Standard sandboxed container
services:
  app:
    image: my-task-base:1.0
```

Reserved Directory Conflicts

Harbor reserves `/logs/verifier/`, `/logs/artifacts/`, `/oracle/`, and `/tests/` for its own use. Creating, modifying, or chowning these paths in the `Dockerfile` will conflict with the runtime mount and cause verifier failures.

DOCKERFILE

[Copy](#)

```
# Bad - Creates or modifies Harbor's reserved paths
RUN mkdir -p /tests /oracle
RUN chown -R appuser:appuser /app /logs /tests /oracle
```

```
# Good - Use a different name for the task's own directories
RUN mkdir -p /app/work /app/output
RUN chown -R appuser:appuser /app
```

Heredocs and Opaque Archives

Embedding source files via `cat <<EOF` heredocs or shipping them inside opaque tarballs/zips makes the Dockerfile hard to review and the resulting image hard to audit. Files should be stored as files and copied normally.

DOCKERFILE

 Copy

```
# Bad - Source code inlined via heredoc
RUN cat > /app/main.py <<'EOF'
def main():
    ...
EOF

# Good - Source lives as a real file, copied normally
COPY app/main.py /app/main.py
```

Oversized Build Context

The `environment/` directory must stay within 100 MiB total and no single file may exceed 50 MiB. Large fixtures, datasets, or accidentally-included `node_modules` / `.git` directories blow this out.

TEXT

 Copy

```
# Bad - Multi-gigabyte fixtures in the build context
environment/
├─ Dockerfile
└─ fixtures/
   ├─ full-database-dump.sql      ← 1.2 GB
   └─ sample-videos/            ← 800 MB
      └─ ...

# Good - Trim fixtures, fetch large data at build time from a pinne
```

```
environment/  
├─ Dockerfile  
├─ fixtures/  
  └─ small-sample.sql          ← 200 KB
```

Unsanctioned or Floating Base Images

The final runtime base image must be [canonical](#) (or non-canonical with a brief justification) and use a specific version tag with a digest — never [latest](#) or another floating tag. Builder stages may use task-appropriate toolchain images, but the final stage is what agents and verifiers run in and is checked by CI.

DOCKERFILE

[Copy](#)

```
# Bad – Floating tag, unknown provenance  
FROM public.ecr.aws/docker/library/python:latest  
  
# Good – Canonical base, specific version tag with digest pin  
FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm@sha256
```

Note: For version-pinning issues specifically (pip/npm packages unpinned, apt versions floating, base image tag missing the digest), the feedback category is [pinning](#), not [environment](#). The two are related but distinct.

Cheating Opportunities

Feedback categories: [expose_answers](#), [test_alignment](#)

Exposing Test Logic

DOCKERFILE

[Copy](#)

```
# Bad – Agent can read test files
COPY tests/ /tests/
```

Tests should be mounted at runtime, not baked into the image.

Answer in Git History

DOCKERFILE

[Copy](#)

```
# Bad – Agent could see newer commits with answers
RUN git clone https://github.com/example/repo.git

# Good – Pin to specific commit
RUN git clone https://github.com/example/repo.git \
    && cd repo && git checkout abc123
```

Editable Data Files

If tests check data files, ensure agents can't just edit them to pass:

PYTHON

[Copy](#)

```
# Bad – Agent could just write the expected values
def test_processed_data():
    data = json.load(open("/data/output.json"))
    assert data["total"] == 42

# Good – Verify computation, not just final value
def test_computation():
    input_data = json.load(open("/data/input.json"))
    output_data = json.load(open("/data/output.json"))
    expected = sum(item["value"] for item in input_data)
    assert output_data["total"] == expected
```

Difficulty Issues

Feedback category: [task_difficulty](#)

Too Easy

Signs a task might be too easy:

- Single-step solution
- Common tutorial topic
- Simple API usage
- Pattern matching suffices

Too Hard / Unfair

Signs a task might be problematic:

- Requires unavailable information
- Environment is unreliable
- Success depends on luck
- Requirements are contradictory

Quick Reference

CATEGORY	RED FLAG	FIX
Instructions	Relative paths	Use absolute paths
Instructions	"Better", "properly"	Specify requirements
Tests	String matching	Compare objects
Tests	No docstrings	Add docstrings
Tests	Implementation checks	Believe the user
Solution	Echo answers	Delete answers
Solution	Random without seed	Add seed
Environment	Missing <code>tmux</code> / <code>asciinema</code>	Preinstall

CATEGORY	RED FLAG	FIX
Environment	Runtime network installs in <code>test.sh</code>	Bal
Environment	AI-scaffolding filenames (<code>CLAUDE.md</code> , <code>skills.md</code> , etc.)	Re
Environment	<code>solution/</code> or <code>tests/</code> copied in Dockerfile	Use
Environment	<code>--privileged</code> / <code>SYS_ADMIN</code> / docker socket	Use
Environment	Floating base image tag (<code>latest</code>)	Spe
Cheating	Tests in image	Mo
Cheating	Mutable data	Ver

See Also

- [Quality Guidelines](#) — Additional quality standards for TBench 2.0 tasks
- [Reviewer Checklist — Environment](#) — Formal High-severity criteria for Dockerfile/container issues
- [Prompt Styling](#) — Detailed rules behind the `instruction_styling` and `expose_answers` categories
- [Writing Oracle Solution](#) — Detailed rules behind the `oracle` category
- [Writing Tests](#) — Detailed rules behind the `test_alignment` and `test_build` categories

PREVIOUS

[Long Context Checklist](#)

NEXT

Defending Your Submission